

Domain Driven Design

Building Microservice Systems

Sharif University of Technology, Spring 2026

Ali Abrishami

Seyyed Hossein Mirhosseini



Introduction to Domain-Driven Design



Introduction

- It's not uncommon for us to have to develop software for a diverse range of business domains:
 - Financial systems,
 - Medical software,
 - Online retailers,
 - Marketing,
 - and many others.
- Making sense of new **business domains** is very challenging.
- But, even the most complex systems are inherently simple when viewed from the **right angle**.



Software Projects Failure

- According to studies, approximately 70% of software projects are not delivered on **time**, on **budget**, or according to the **client's requirements**.
- The vast majority of software projects **fail**.
- This issue is so deep and widespread that we even have a term for it: ***software crisis***.



The Root Cause!

- Although researchers have not been able to pinpoint a single cause, most of their findings share a common theme: **communication**.
- Communication issues thwarting projects can manifest themselves in different ways:
 - Unclear requirements,
 - Uncertain project goals,
 - Or ineffective coordination of effort between teams.



Effective communication is the central theme of the domain-driven design tools and practices.



The Need for DDD

- **Business logic** is the heart of software. But traditional designs focus on technical structure instead of business logic.
- Software developers are not the only people involved in building software. The **domain experts**, for whom the software is being built, bring critical understanding of the problems being solved.
- Domain-driven design provides a **set of practices** for a collaborative approach to building software from the **perspective of the business** (That is, the **domain**, and its problems).



6 Key Roles in the Software Development Team

Business
Analyst

Project
Manager

UI/UX
Designer

Software
Developer

QA
Engineer

DevOps
Engineer



Two Stages of DDD

- DDD is divided into two stages: strategic and tactical.
- First Stage: “**strategic design**”
 - Understanding the **business problem**, a.k.a. the *domain*; and break the problem down into smaller, solvable, interconnected problems.
 - Communicate in the **language of the domain**.
- Second Stage: “**tactical design**”
 - In this stage we **transform** the discoveries of strategic design into **software** architecture and implementation.
 - DDD provides guidance and patterns for organizing these domains and avoiding further complexity.



DDD for Software Engineers



- DDD provide tools and techniques for modeling and effectively implementing software.
- It also illuminates an often-overlooked aspect of software engineering: the **context**.
- Equipped with the knowledge of the system's business problem, you will be much more effective at choosing the **appropriate solution**.
- A solution that is not under- or over-engineered, but addresses **business needs** and **goals**.



DDD for Software Architects



- DDD is even more important for software architects, and even more so for aspiring software architects.
- Its strategic design decision tools will help you decompose a large system into **components**—services, microservices, or subsystems—and design how the components are **integrated** with one another to form a system.



Analyzing Business Domains



Aligning Software with Business Strategy

- The strategic aspect of DDD deals with answering the questions of “what?” and “why?”.
 - *What software we are building and why we are building it.*
- To design and build an effective solution, you have to understand the **problem** (the **software system** we have to build).
- To understand the problem, you have to understand the **context** within which it exists.
 - The organization’s business strategy, and what value it seeks to gain by building the software.



What Is a Business Domain?

- A business domain defines a company's **main area of activity**.
- Generally speaking, it's **the service** the company provides to its clients.
 - *FedEx* provides courier delivery.
 - *Starbucks* is best known for its coffee.
 - *Walmart* is one of the most widely recognized retail establishments.
- A company can operate in multiple business domains.



What Is a Subdomain?

- To achieve its business domain's goals and targets, a company has to operate in multiple *subdomains*.
- A subdomain is a **fine-grained** area of business activity. And all of a company's subdomains form its business domain.
- The subdomains have to interact with each other to achieve the company's goals in its business domain.
- All of them together are necessary for a company to be able to compete in its business domain(s).



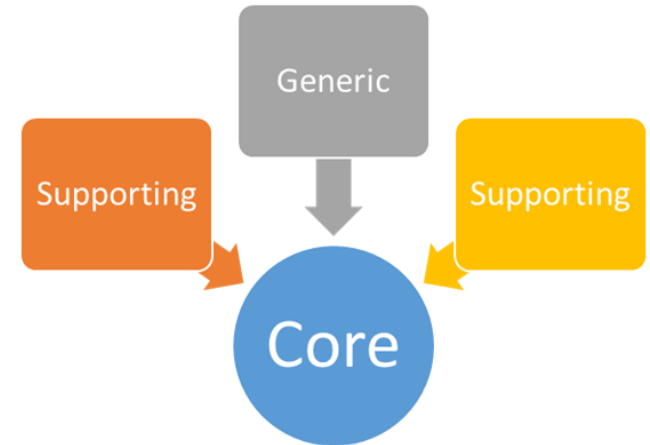
Subdomain Example

- *Starbucks* may be most recognized for its coffee.
- But building a successful coffeehouse chain requires to:
 - Buy or rent real estate at effective locations,
 - Hire personnel,
 - Manage finances,
 - And, other activities.



Types of Subdomains

- Subdomains bear different strategic/business values.
- Domain-driven design distinguishes between three types of subdomains:
 - 1) Core
 - 2) Generic
 - 3) Supporting



Core Subdomains

- A **core subdomain** is what a company does differently from its competitors.
- This may involve inventing new products or services or reducing costs by optimizing existing processes.
- This is the company's strategy for providing better service to its customers and/or maximizing its profitability.



Core Subdomain Example

- Example: *Uber*.
 - The company provided a novel form of transportation: **ridesharing**.
 - Reducing costs by matching riders heading in the same direction.
- Example: *Google Search's ranking algorithm*.
 - Google's advertising platform accounts for the majority of its profits.
 - Although the search engine is not a paid service, it serves as the largest display platform for *Google Ads*.



Analyzing Core Subdomains

- **Complexity:**

- A core subdomain that is simple to implement can only provide a **short-lived** competitive advantage.
- Core subdomains are naturally **complex**.
- It should be hard for competitors to copy or imitate the company's solution.

- **Sources of competitive advantage:**

- Core subdomains are not necessarily technical.
- For example, a jewelry maker selling its products online.
- The online shop is important, but it's not a core subdomain. The **jewelry design** is.



Generic subdomains

- **Generic subdomains** are business activities that all companies are performing in the same way.
- Like core subdomains, generic subdomains are generally **complex** and **hard** to implement.
- However, generic subdomains do not provide any competitive edge for the company.
- For example, most systems need to authenticate and authorize their users.
- In jewelry example, the **online shop** is a generic subdomain. Using the same online retail platform as its competitors would not impact the jewelry maker's competitive advantage.



Supporting subdomains

- **Supporting subdomains** support the company's business.
- However, contrary to core subdomains, supporting subdomains **do not** provide any competitive advantage.
- Supporting subdomains are **simple**.
- Their business logic resembles mostly data entry screens and **ETL operations** (extract, transform, load); that is, the so-called **CRUD interfaces** (create, read, update, delete).
- These activity areas do not provide any competitive advantage for the company.

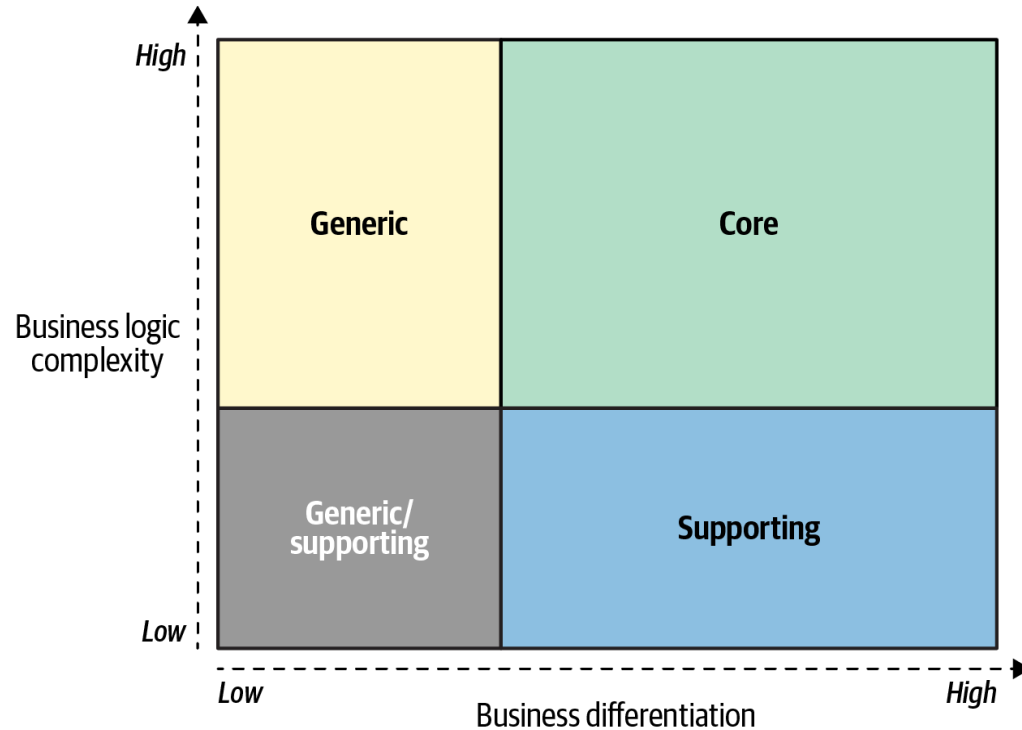


Comparing Subdomains

- Competitive advantage:
 - Only **core subdomains** provide a competitive advantage to a company.
 - The **more complex** the problems a company is able to tackle, the more **business value** it can provide.
- Complexity:
 - Supporting subdomains' business logic is simple. These are basic ETL operations and CRUD interfaces, and the business logic is obvious.
 - Generic subdomains are much more complicated.
 - Core subdomains are **complex**. They should be as hard for competitors to copy as possible (the company's profitability depends on).



Comparing Subdomains (cont.)



Comparing Subdomains (cont.)

- Volatility
 - Core subdomains can change often.
 - If a problem can be solved on the first attempt, it's probably not a good competitive advantage.
 - Companies **continuously** innovate and evolve core subdomains.
 - Supporting subdomains do not change often.
 - Generic subdomains can change over time.



Solution strategy

- The subdomains are like foundational building blocks: take one away and the whole structure may fall down.
- Core subdomains have to be implemented **in-house**.
They cannot be bought or adopted.
And it would also be unwise to outsource the implementation of a core subdomain.
- The organization's **most skilled talent** should be assigned to work on its **core** subdomains.



Solution strategy (cont.)

- It's more cost effective to buy an off-the-shelf product or adopt an open-source solution than invest time and effort into implementing a **generic** subdomain in-house.
- A rapid application development framework will suffice to implement **supporting** subdomains.
- Save the engineers on your team who are experienced in tackling complex challenges for the **core** subdomains.



Comparison

Subdomain type	Competitive advantage	Complexity	Volatility	Implementation	Problem
Core	Yes	High	High	In-house	Interesting
Generic	No	High	Low	Buy/adopt	Solved
Supporting	No	Low	Low	In-house/outsource	Obvious

The differences between the three types of subdomains



Example: Gigmaster



- *Gigmaster* is a ticket sales and distribution company. Its mobile app analyzes users' music libraries, streaming service accounts, and social media profiles to identify nearby shows that its users would be interested in attending.
- *Gigmaster's* users are conscious of their privacy. Hence, all users' personal information is encrypted. Moreover, to ensure that users' guilty pleasures won't leak out under any circumstances, the company's recommendation algorithm works exclusively on anonymized data.
- To improve the app's recommendations, a new module was implemented. It allows users to log gigs they attended in the past, even if the tickets weren't purchased through *Gigmaster*.



Example: Gigmaster (cont.)



- Gigmaster's *business domain* is ticket sales.
- Gigmaster's *core subdomains* are:
 - Recommendation engine
 - Data anonymization
 - Mobile app
- Gigmaster's *generic subdomains* are:
 - Encryption, for encrypting all data
 - Accounting, since the company is in the sales business
 - Clearing, for charging its customers
 - Authentication and authorization, for identifying its users



Example: Gigmaster (cont.)



- Gigmaster's *supporting* subdomains (ETL processes or CRUD interfaces):
 - Integration with music streaming services
 - Integration with social networks
 - Attended-gigs module



Importance of Finding Subdomains

- **Reduce system complexity** by dividing the domain into smaller, manageable parts.
- **Identify business priorities** and focus effort on the most valuable areas (core subdomain).
- **Establish clear system boundaries** that support modular and maintainable architecture.
- **Align software design with business capabilities** for better communication with domain experts.
- **Enable efficient team organization** by assigning ownership of different subdomains.



Who Are the Domain Experts?

- **Domain experts** are subject matter experts who know all the intricacies of the business that we are going to model and implement in code.
- Domain experts are **knowledge authorities** in the software's business domain.
- As a rule of thumb, domain experts are either the people coming up with requirements or the software's end users.
- The software is supposed to **solve** their problems.
- They have **in-depth** knowledge of the company's business domain or one or more of its subdomains and are **critical** to a project's success.



Discovering Domain Knowledge

It's developers' (mis)understanding, not domain experts' knowledge, that gets released in production. (Alberto Brandolini)



Business Problems

- The software systems we are building are solutions to business problems.
- A business problem can be challenges associated with:
 - optimizing workflows and processes,
 - minimizing manual labor,
 - supporting decisions,
 - managing resources,
 - managing data,
 - and so on.



Knowledge Discovery

- The software has to mimic the domain experts' way of thinking about the problem.
- A software project's success depends on the effectiveness of **knowledge sharing** between domain experts and software engineers.
- We have to understand the problem in order to solve it.
- Effective knowledge sharing between domain experts and software engineers requires **effective communication**.

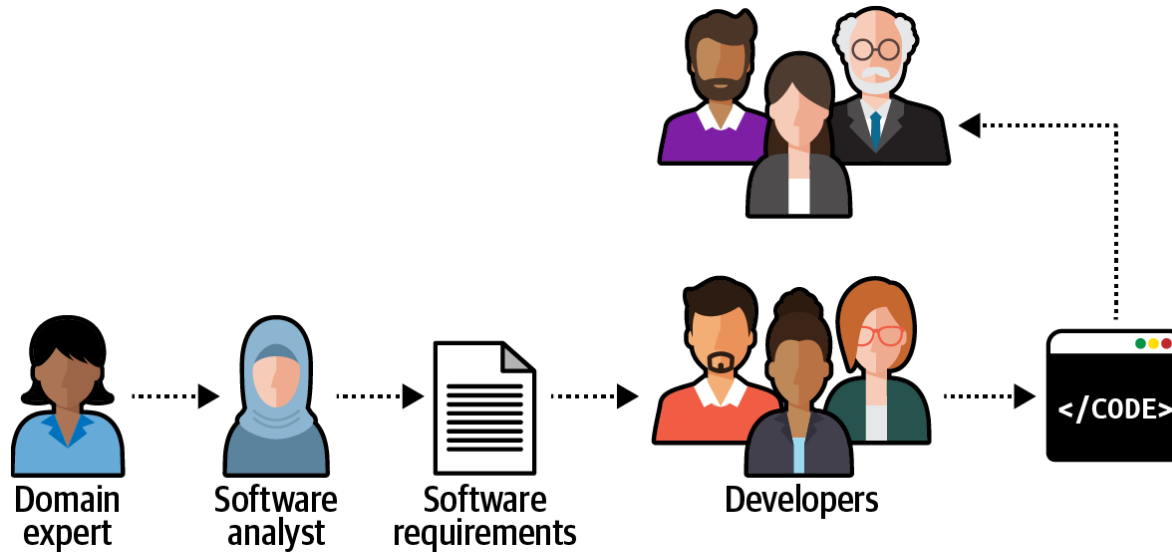


Communication

- Almost all software projects require the collaboration of stakeholders in different roles: *domain experts, product owners, engineers, UI and UX designers, project managers, testers, analysts, and others.*
- Research into why software projects fail has shown that **effective communication** is essential for knowledge sharing and project success.



Communication (cont.)

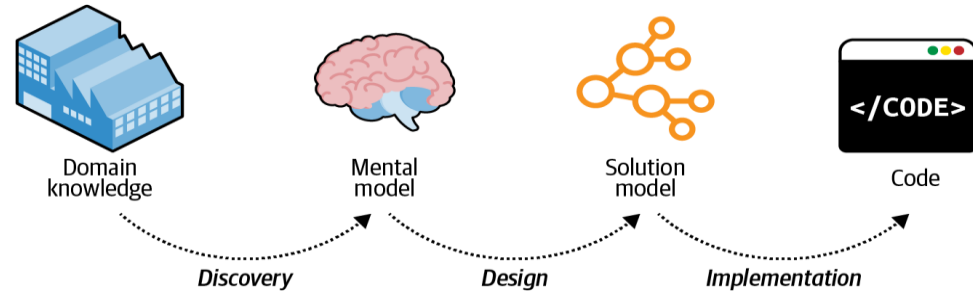


Knowledge sharing flow in a software project



Communication (cont.)

- The traditional software development lifecycle implies the following translations:
 - Domain knowledge into an analysis model,
 - Analysis model into requirements,
 - Requirements into system design,
 - System design into source code.



Knowledge sharing flow in a software project



Communication (cont.)

- In traditional software development, the domain knowledge is “translated” into an **engineer-friendly** form.
 - Known as an *analysis model*.
- Such mediation is hazardous to knowledge sharing.
 - In any translation, **information is lost**.
- Such a software development process resembles the *children’s game Telephone*.
- The last player receives a message that is significantly different from the original one.



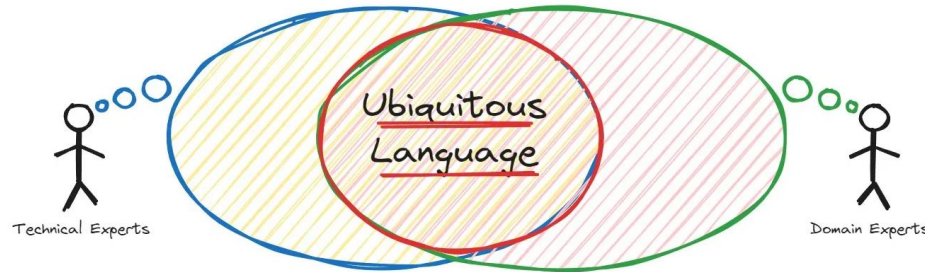
What Is a Ubiquitous Language?

- The idea is simple and straightforward:
*If parties need to communicate efficiently, instead of relying on translations, they have to speak the **same language**.*
- Domain-driven design calls for cultivating a **single language** for describing the business domain: the **ubiquitous language**.
- All project-related stakeholders should use the ubiquitous language when describing the business domain.
- Ubiquitous language is an effective tool for **bridging the knowledge gap** between domain experts and software engineers.



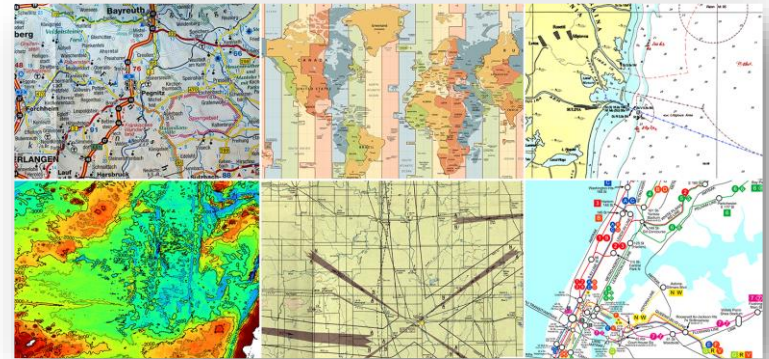
Language of the Business

- The ubiquitous language aims to frame the domain experts' **understanding** and **mental models** of the business domain in terms that are easy to understand.
- The ubiquitous language must be **precise** and **consistent**.
- Each term of the ubiquitous language should have **one and only one** meaning.
- Two terms cannot be used interchangeably in a ubiquitous language.



Model of the Business Domain

- A model is a **simplified** representation of a thing or phenomenon that intentionally emphasizes **certain aspects** while ignoring others. Abstraction with a specific use in mind. (Rebecca Wirfs-Brock)
- A model is **not** a copy of the real world but a human construct that helps us make sense of real-world systems.



Different types of maps displaying different models of the earth



Effective Modeling

- All models have a purpose, and an effective model contains only the details needed to fulfill its purpose.
- In its essence, a model is **an abstraction**.
- The model is supposed to capture the **domain experts' mental models**— their thought processes about how the business works to implement its function.
- The model is supposed to include just **enough** aspects of the business domain to make it possible to implement the required system.



Forming Ubiquitous Language

- Formulation of a ubiquitous language requires interaction with its natural holders, the *domain experts*.
- Cultivation of a ubiquitous language is an ongoing process. It should be constantly **validated** and **evolved**.
- A **wiki** can be used as a *glossary* to capture and document the ubiquitous language.
- When a ubiquitous language is changed, **all** team members should be encouraged to go ahead and update the glossary.
- There are even static code analysis tools that can verify the usage of a ubiquitous language's terms. A notable example for such a tool is *NDepend*.



Ubiquitous Language Example

- Food Delivery Platform:
 - A **Customer** places an **Order** from a **Restaurant** using the platform.
 - The restaurant **accepts** the order and **prepares** the food.
 - A **Courier** is assigned to **deliver** the order.
 - The customer **pays** for the order and can **track** the delivery.



Ubiquitous Language Example (cont.)

- **Core Entities**
 - Customer, Restaurant, Courier, Order, MenuItem
- **Value Objects**
 - Address, Money, OrderItem, DeliveryTimeEstimate
- **Aggregates**
 - Order (contains OrderItems, OrderStatus, PaymentStatus, DeliveryAddress)
- **Domain Actions**
 - PlaceOrder, AcceptOrder, PrepareOrder, AssignCourier, PickupOrder
- **Domain Rules**
 - A Customer places an Order.
 - A Restaurant accepts an Order.
 - An Order contains OrderItems.



Managing Domain Complexity



Inconsistent Models

- We develop a ubiquitous language that can be used for communication by all stakeholders, from software engineers to domain experts.
- The language should reflect the domain experts' **mental models** of the **business** domain's inner workings and underlying principles.
- The language must be clear and consistent. It should be free of ambiguity, implicit assumptions, and extraneous details.
- However, on an **organizational scale**, the domain experts' mental models can be **inconsistent** themselves.



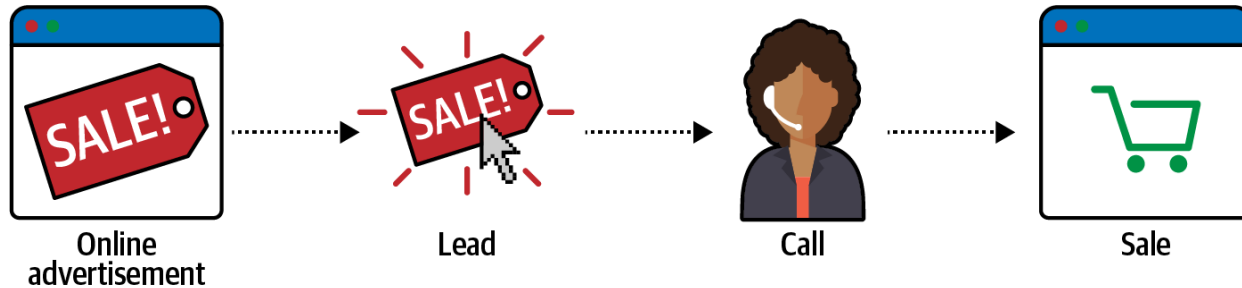
Inconsistent Models Example

- The company's marketing department generates leads through online advertisements.
- But the term *lead* has different meanings in the marketing and sales departments:
 - *Marketing department*
 - For the marketing people, a lead represents a notification that somebody is interested in one of the products.
 - *Sales department*
 - It represents the entire lifecycle of the sales process. It's not a mere event, but a long-running process.



Inconsistent Models Example (cont.)

- In this case, the mental model of the “lead” is inconsistent among the domain experts in the *sales* and *marketing* departments.
- Communication can be more challenging among people from different departments.

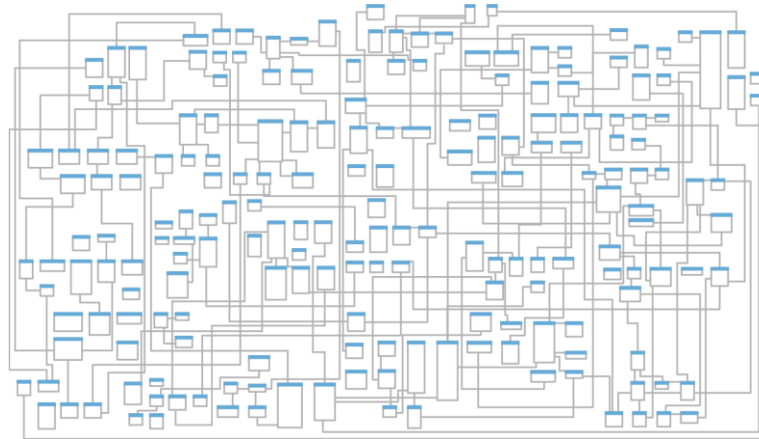


Example business domain: telemarketing company



Some Solutions

- How do we solve this catch?
- The traditional solution to this problem is to design a **single model** that can be used for all kinds of problems.



Enterprise-wide entity relationship diagram



Some Solutions (cont.)

- *“jack of all trades, master of none.”*
- Such models are supposed to be suitable for everything but eventually are effective for nothing.
- No matter what you do, you are always facing complexity:
 - The complexity of filtering out extraneous details,
 - The complexity of finding what you do need,
 - And most importantly, the complexity of keeping the data in a consistent state.



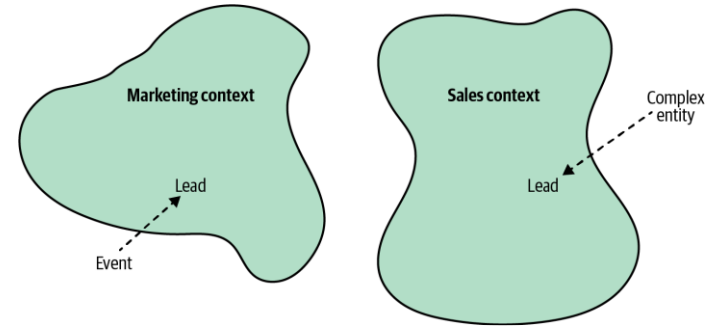
Some Solutions (cont.)

- Another solution would be to prefix the problematic term with a definition of the context: “marketing lead” and “sales lead.”
- However, this approach has two main disadvantages:
 - First, it induces **cognitive load**. When should each model be used? The closer the implementations of the conflicting models are, the easier it is to make a mistake.
 - Second, the implementation of the model won't be aligned with the ubiquitous language (No one would use the prefixes in conversations).



What Is a Bounded Context?

- The solution in domain-driven design:
 1. Divide the ubiquitous language into multiple smaller languages,
 2. Then assign each one to the explicit context in which it can be applied: its ***bounded context***.
- In the preceding example, we can identify two bounded contexts: *marketing* and *sales*.
 - Each fine-grained ubiquitous language is consistent and follows the domain experts' mental models.



splitting it into bounded contexts



Model Boundaries

- As previously discussed, a model is **not** a copy of the real world but a construct that helps us make sense of a complex system.
- That makes defining a **model's boundary**—its bounded contexts—an intrinsic part of the modeling process.
- A ubiquitous language in one bounded context can be completely **irrelevant** to the scope of **another** bounded context.
- A language's terminology, principles, and business rules are only consistent inside **its** bounded context.



Ubiquitous Language Refined

- A ubiquitous language is **not universal**.
- A ubiquitous language is ubiquitous only in the **boundaries** of its bounded context.
- The language is focused on describing only the model that is encompassed by the bounded context.



Scope of a Bounded Context

- Defining the scope of a ubiquitous language—its bounded context—is a **strategic** design decision.
- The wider the boundary of the ubiquitous language is, the harder it is to keep it consistent.
- It may be beneficial to divide a large ubiquitous language into smaller, more manageable problem domains, but striving for small bounded contexts can backfire too.



Scope of a Bounded Context (cont.)

- The reasons for extracting finer-grained bounded contexts out of a larger one include:
 - Constituting new software engineering teams or,
 - Addressing some of the system's nonfunctional requirements.
- One thing to beware of is splitting a coherent functionality into multiple bounded contexts.
 - Such division will hinder the ability to evolve each context independently.
 - Instead, the same business requirements and changes will simultaneously affect the bounded contexts and require simultaneous deployment of the changes.

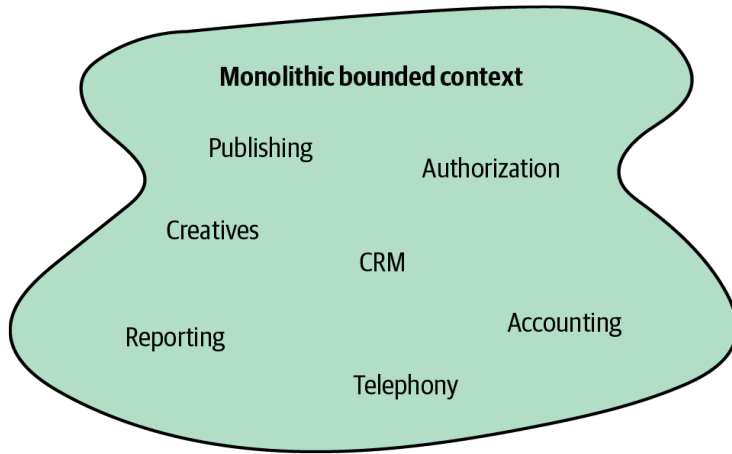


Bounded Contexts Versus Subdomains (1)

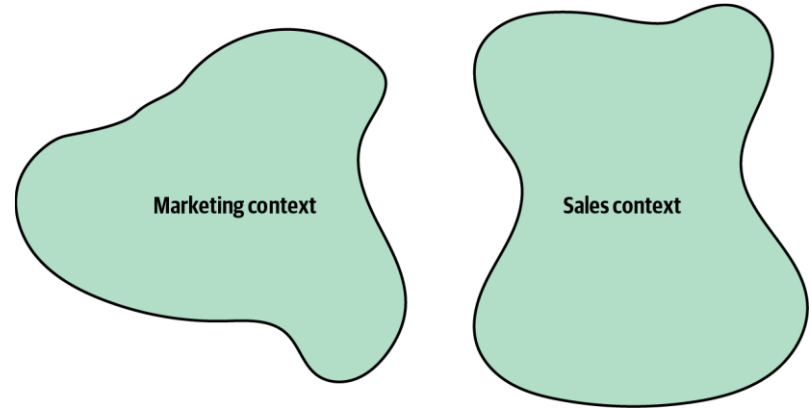
- Subdomains:
 - To comprehend a company's **business strategy**.
 - The analysis phase involves identifying the different subdomains.
 - That's how the organization **works** and plans its competitive strategy.
- Bounded Contexts:
 - Bounded contexts, on the other hand, are designed.
 - Choosing models' boundaries is a **strategic** design decision.
 - We decide how to **divide** the business domain into smaller, manageable problem domains.
- It's crucial to remember that subdomains are **discovered** and bounded contexts are **designed**.



Bounded Contexts Versus Subdomains (2)



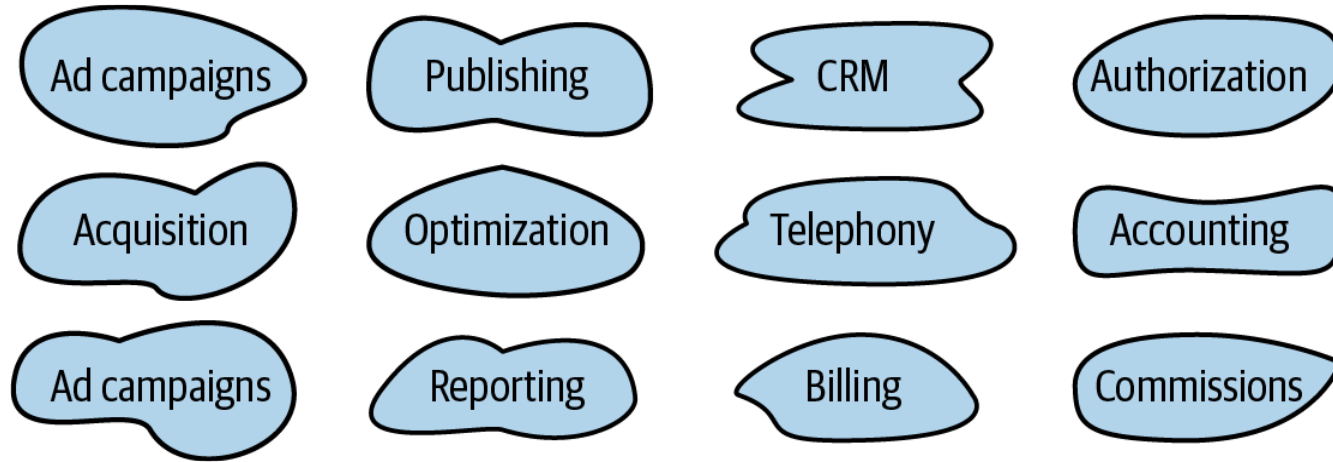
1) Monolithic bounded context



2) Bounded contexts driven by the consistency of the ubiquitous language



Bounded Contexts Versus Subdomains (2)



3) Bounded contexts aligned with subdomains' boundaries



Bounded Contexts Versus Subdomains (3)

Practical Rule Used by Most DDD Teams:

1. Identify **subdomains** (business capability).
2. Break them into **bounded contexts** (model boundaries).
3. Implement each bounded context as a **microservice**.



Boundaries

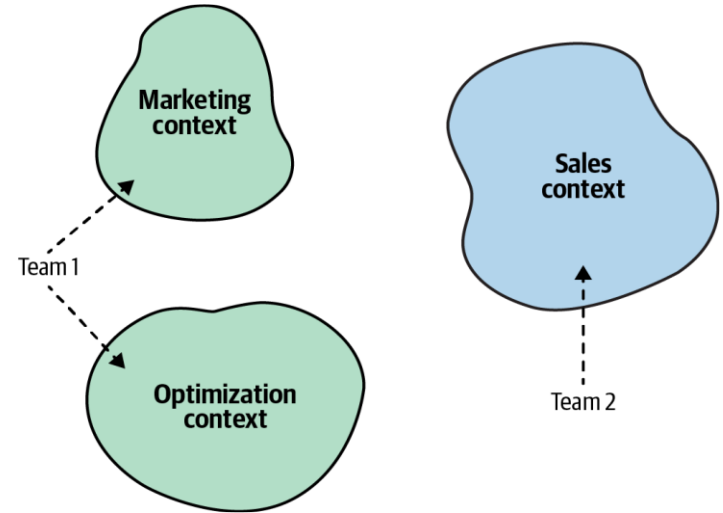
- As Ruth Malan says, ***architectural design is inherently about boundaries.***
- The bounded context pattern is the domain-driven design tool for defining **physical** and **ownership boundaries**.
- Physical Boundaries:
 - Bounded contexts serve not only as model boundaries but also as physical boundaries of the systems implementing them.
 - Each bounded context should be implemented as an individual service/project, meaning it is **implemented**, **evolved**, and **versioned** independently of other bounded contexts.



Boundaries (cont.)

- Ownership Boundaries:

- In software projects, we can leverage model boundaries—bounded contexts—for the peaceful **coexistence of teams**.
- A bounded context should be implemented, evolved, and maintained by **one team** only.
- No two teams can work on the same bounded context.



Example of bounded context ownership



Integrating Bounded Contexts



Integrating Bounded Contexts

- You cannot build a model without specifying its boundary.
- The boundary divides the responsibility of languages.
- A language in one bounded context can model the business domain to solve a particular problem.
- Another bounded context can represent the same business entities but model them to solve a different problem.



Integrating Bounded Contexts (cont.)

- Although the bounded contexts can evolve independently, they have to **integrate** with one another.
- As a result, there will always be **touchpoints** between bounded contexts. These are called **contracts**.
- The need for contracts results from differences in bounded contexts' models and languages.
- Since each contract affects more than one party, they need to be defined and **coordinated**.
- In this section, we will learn about domain-driven design patterns for defining **relationships** and **integrations** between bounded contexts.



Integrating Bounded Contexts (cont.)

- These patterns are driven by the nature of collaboration between teams working on bounded contexts.
- Patterns are divided into three groups, each representing a type of team collaboration:
 1. Cooperation
 1. Partnership
 2. Shared kernel
 2. Customer–supplier
 1. Conformist
 2. Anticorruption layer
 3. Open-host service
 3. Separate ways



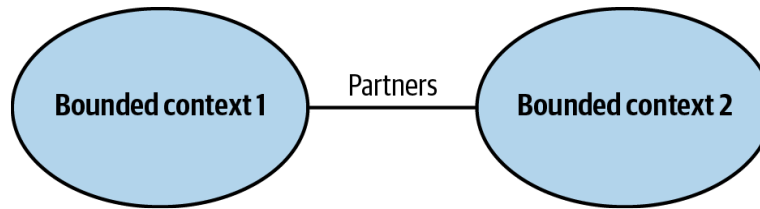
1) Cooperation

- Cooperation patterns relate to bounded contexts implemented by teams with **well established communication**.
- In the simplest case, these are bounded contexts implemented by a single team.
- This also applies to teams with dependent goals, where one team's success depends on the success of the other, and vice versa.
- Two DDD patterns suitable for cooperating teams:
 - Partnership patterns
 - Shared kernel patterns



1-1) Partnership Model

- In the partnership model, the integration between bounded contexts is coordinated in an ad hoc manner.
- One team can notify a second team about a change in the API, and the second team will cooperate and **adapt**—no drama or conflicts.



The partnership model



1-1) Partnership Model (cont.)

- The coordination of integration here is **two-way**. **No one team dictates the language** that is used for defining the contracts.
- Continuous integration of the changes applied by both teams is needed.
- So, successful integration depends on:
 - Well-established collaboration practices
 - High levels of commitment
 - Frequent synchronizations between teams
- Not a good fit for geographically distributed teams since it may present synchronization and communication challenges.



1-1) Partnership Model Example

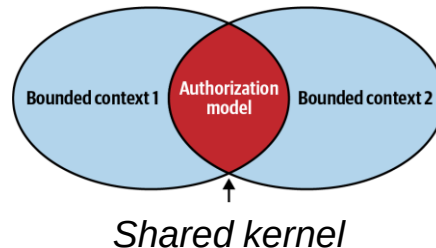
- A company builds a logistics platform that has two bounded contexts:
 - **Shipping Context** – responsible for planning and executing shipments.
 - **Billing Context** – responsible for invoicing and payment processing.

Step	Context	Operation
1	Shipping	Customer creates a shipment order
2	Shipping → Billing	Calls Billing API /authorize-payment
3	Billing	Validates account balance, returns authorization token
4	Shipping	Stores authorization, marks shipment as confirmed
5	Billing	Later listens for “shipment completed” event to issue invoice



1-2) Shared Kernel

- There can be cases when the same model of a subdomain, or a part of it, will be implemented in **multiple** bounded contexts.
- The *shared model* is designed according to the needs of all of the bounded contexts.
- **Example:** Consider an enterprise system that uses a tailor-made model for managing users' permissions. Each user can have their permissions granted directly or inherited from one of the organizational units they belong to.



1-2) Shared Kernel (cont.)

- A change made to the shared model has an immediate effect on **all** the bounded contexts.
- The shared kernel will consist only of **integration contracts** and **data structures** that are intended to be passed across the bounded contexts' boundaries.
- If the organization uses the mono-repository approach, these can be the same source files referenced by multiple bounded contexts.
- The shared kernel can be extracted into a dedicated project and referenced in the bounded contexts as a *linked library*.



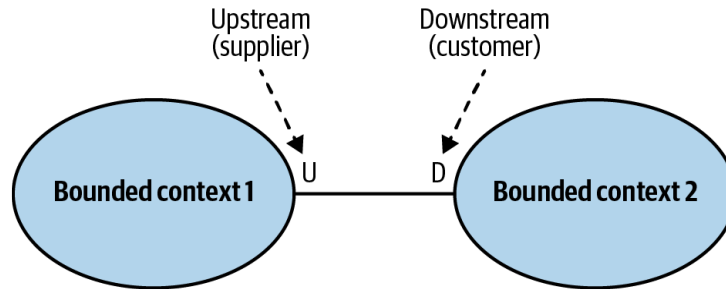
1-2) Shared Kernel Use Cases

- The shared kernel will naturally be applied for the subdomains that change the most: the **core subdomains**.
- Another common use case for applying the shared kernel pattern, albeit a temporary one, is the **gradual modernization of a legacy system**.
- In such a scenario, the shared codebase can be a pragmatic intermediate solution for gradually decomposing the system into bounded contexts.



2) Customer–Supplier

- One of the bounded contexts—***the supplier***—provides a service for its customers.
- The service provider is “**upstream**” and the customer or consumer is “**downstream**”.



Customer–supplier relationship



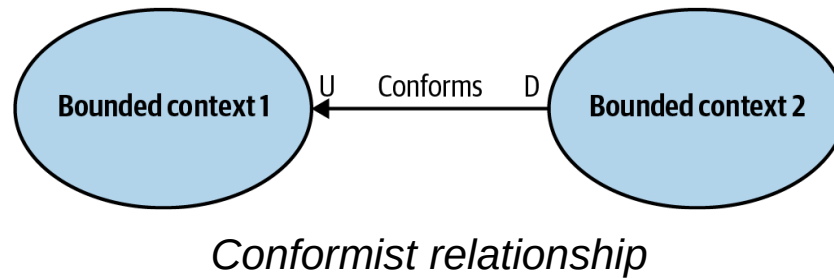
2) Customer–Supplier (cont.)

- Both teams (upstream and downstream) can succeed independently.
- Consequently, in most cases we have an imbalance of power: **either** the upstream or the downstream team can dictate the integration contract.
- Three patterns addressing such power differences:
 1. Conformist
 2. Anticorruption layer
 3. Open-host service



2-1) Conformist

- The balance of power favors the **upstream** team.
- It just provides the integration contract, defined according to its own model—take it or leave it.
- If the downstream team can accept the upstream team's model, the bounded contexts' relationship is called **conformist**.



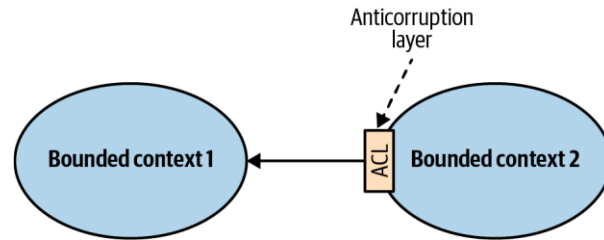
2-1) Conformist (cont.)

- The downstream conforms to the upstream bounded context's model.
- The contract exposed by the upstream team may be:
 - An industry-standard,
 - Well-established model,
 - Or it may just be good enough for the downstream team's needs.



2-2) Anticorruption Layer

- As in the conformist pattern, the balance of power in this relationship is still skewed toward the **upstream** service.
- However, in this case, the downstream bounded context is not willing to conform.
- It can translate the upstream bounded context's model into a model tailored to its own needs via an ***anticorruption layer***.



Integration through an anticorruption layer



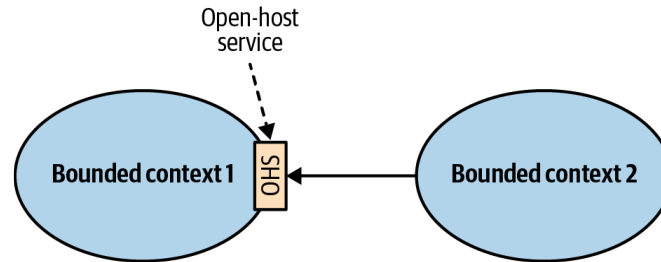
2-2) Anticorruption Layer Use Cases (cont.)

1. *When the downstream bounded context contains a core subdomain*
 - Adhering to the supplier's model might **impede** the modeling of the problem domain of core subdomain.
2. *When the upstream model is inefficient or inconvenient for the consumer's needs*
 - If a bounded context conforms to a mess, it risks becoming a mess itself.
 - That is often the case when integrating with **legacy systems**.
3. *When the supplier's contract changes often*
 - The consumer wants to protect its model from **frequent changes**.
 - With an anticorruption layer, the changes in the supplier's model only affect the translation mechanism.



2-3) Open-Host Service

- This pattern addresses cases in which the power is skewed toward the **consumers**.
- The supplier is interested in **protecting its consumers** and providing the best service possible.
- The upstream supplier decouples the implementation model from the public interface.



Integration through an open-host service



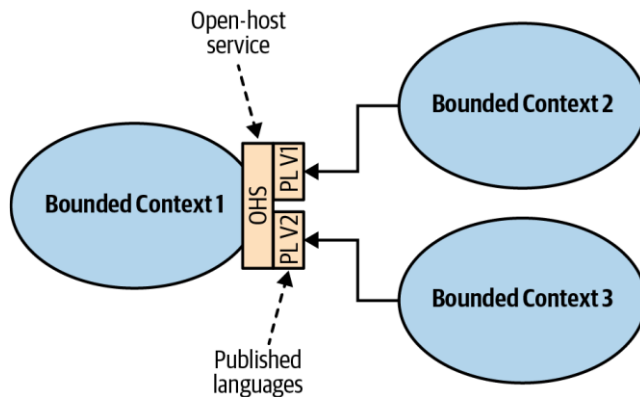
2-3) Open-Host Service (cont.)

- The supplier's public interface is **not** intended to conform to its ubiquitous language.
- Instead, it is intended to expose **a protocol convenient for the consumers**, expressed in an integration-oriented language.
- As such, the public protocol is called the ***published language***.
- The open-host service pattern is a reversal of the anticorruption layer pattern.
 - Instead of the consumer, the supplier implements the translation of its internal model.



2-3) Open-Host Service (cont.)

- Furthermore, the integration model's decoupling allows the upstream bounded context to simultaneously expose multiple versions of the published language, allowing the consumer to migrate to the new version gradually



Open-host service exposing multiple versions of the published language.



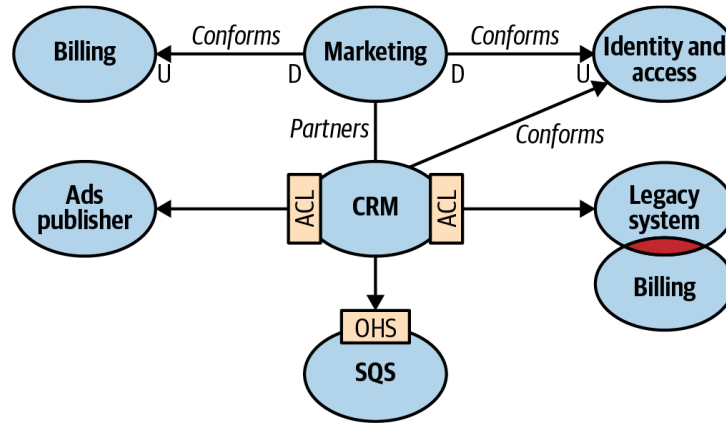
3) Separate Ways

- The last collaboration option is not to collaborate at all :)
- In cases where the teams are **unwilling** or **unable** to collaborate.
- When teams have a hard time collaborating and agreeing, it may be more **cost-effective** to go their **separate ways** and **duplicate** functionality in multiple bounded contexts.
- For example, when the subdomain in question is generic, and if the generic solution is easy to integrate.
 - An example is a logging framework; it would make little sense for one of the bounded contexts to expose it as a service.



Context Map

- After analyzing the integration patterns between a system's bounded contexts, we can plot them on a **context map**.



Context map



Context Map (cont.)

- The context map is a **visual** representation of the system's bounded contexts and the **integrations** between them.
1. High-level design
 - A context map provides an overview of the system's **components** and the models they implement.
 2. Communication patterns
 - A context map depicts the communication patterns among teams.
 3. Organizational issues
 - A context map can give insight into organizational issues.



Context Map (cont.)

- Ideally, a context map should be introduced into a project **right from the get-go**.
- Each team is responsible for updating its own integrations with other bounded contexts.
- A context map can be managed and maintained as code, using a tool like ***Context Mapper***.



Identifying Subdomain Boundaries

1. Event Storming

- Run a workshop where domain experts map out business events on a timeline. Clusters of events that belong together hint at a subdomain.

2. Linguistic Analysis

- When the same term carries different meanings across teams, that's a boundary signal — this is called a “linguistic fracture line.”



Identifying Subdomain Boundaries (cont.)

3. Organizational Mapping

- Conway's Law says your architecture mirrors your organization chart. Map your teams, and subdomain boundaries often follow.

4. Data Ownership

- Ask: "Who is the source of truth for this data?" Different answers = different subdomains.



References

- Learning Domain-Driven Design, *Vlad Khnonov*.
 - Chapters 1, 2, 3, 4

